

Precondizione $i_count \geq 1$ per la sostituzione $igrab \rightarrow ihold$

Prova formale sulla call chain segnalata da lockdep

Kernel Linux for-next (commit 2ffc6900d5c3) · bug syzbot 5eb0d61dfb76ca12670c

1. Domanda

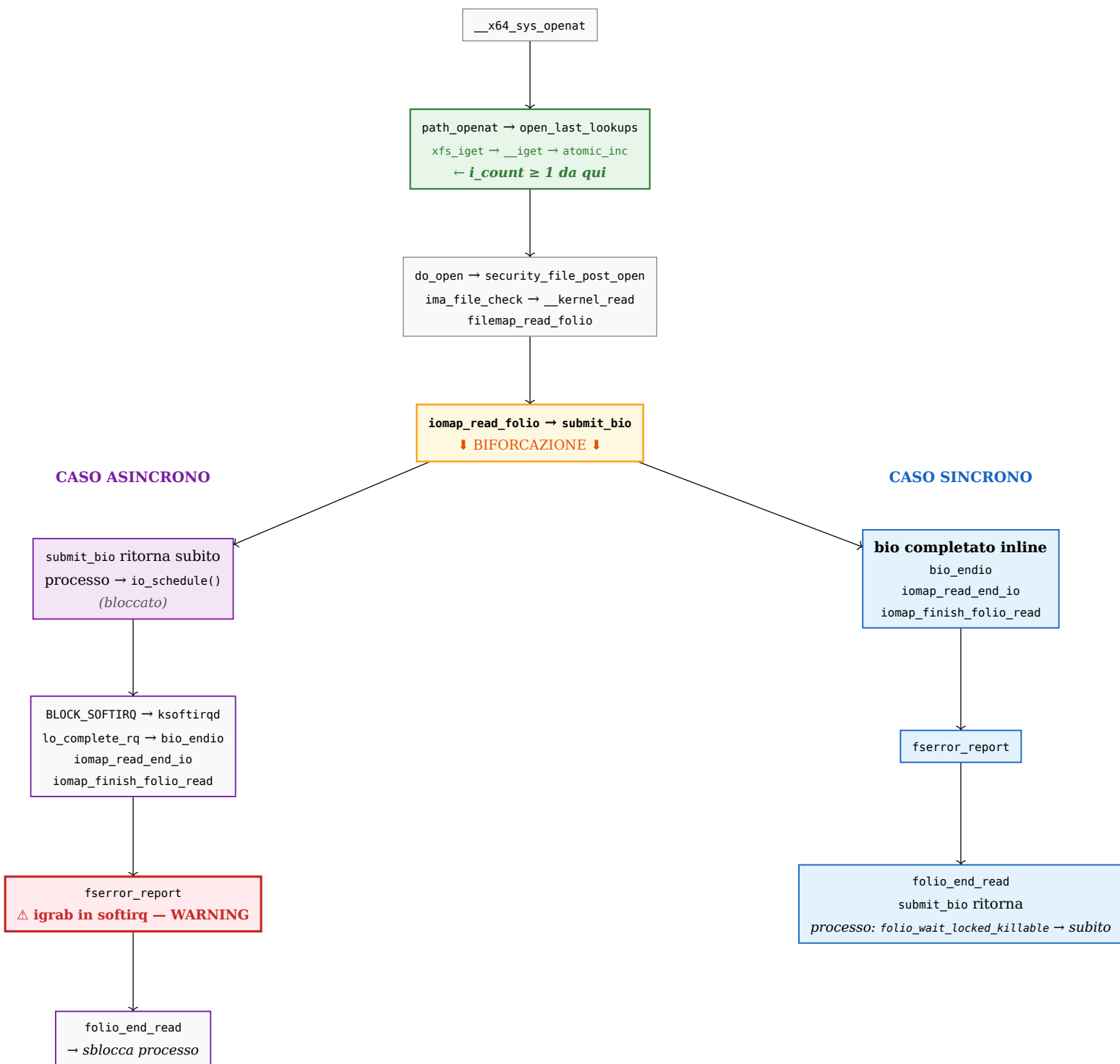
La correzione proposta sostituisce `igrab(inode)` con `ihold(inode)` in `fserror_report()` (`fs/fserror.c:159`). `ihold` incrementa `i_count` senza acquisire `i_lock`, assumendo che esista già almeno un riferimento vivo.

Precondizione di `ihold`: $i_count \geq 1$ al momento della chiamata, e quindi `I_FREEING = I_WILL_FREE = 0`.

Domanda: la call chain segnalata da lockdep garantisce strutturalmente queste condizioni ogni volta che `fserror_report` viene raggiunta?

2. Percorso comune e biforcazione

In entrambi i casi osservati nel log l'esecuzione parte dalla stessa syscall `openat()`. Il percorso fino al block layer è identico; la biforcazione avviene al momento della sottomissione del bio.



2.0.1. L'iget che mantiene i_count ≥ 1

Al momento della risoluzione del path (dentro `open_last_lookups`), prima ancora che il controllo raggiunga IMA o il block layer, `path_openat` acquisisce l'inode tramite XFS. Per XFS, in caso di cache hit, il percorso è:

```
// fs/xfs/xfs_ichache.c:584 (dentro xfs_iget_cache_hit)
error = igrab(inode) ? 0 : -EAGAIN;
```

`igrab` chiama `__iget`, che esegue l'incremento atomico:

```
// include/linux/fs.h:2991
static inline void __iget(struct inode *inode)
{
    lockdep_assert_held(&inode->i_lock);
    atomic_inc(&inode->i_count);    // ← i_count portato a >= 1
}
```

Questo incremento è nel **percorso comune**, prima della biforcazione. Vale per entrambi i casi.

2.0.2. Perché entrambi i casi si verificano con lo stesso loop device

Il loop device può completare un bio in modo **sincrono** (se il backing file è già in page cache → bio_endio viene chiamata prima che submit_bio ritorni) oppure **asincrono** (il completamento viene deferito a ksoftirqd tramite BLOCK_SOFTIRQ). Il comportamento dipende dal timing: non è deterministico. Nel log entrambi i casi si sono materializzati in due iterazioni consecutive del reproducer. Il lockdep warning scatta **solo** nel caso asincrono, perché è lì che igrab viene chiamato in contesto softirq — nel caso sincrono gira in process context e non c'è inconsistenza di locking.

3. Evidenza dal log

Il log contiene due tipi di messaggi con origini diverse: il warning lockdep, prodotto interamente dalla macchina lockdep del kernel senza modifiche al codice, e i messaggi [DBG], aggiunti esplicitamente per catturare lo stato degli inode nel momento critico.

3.1. Messaggi comuni — gli errori I/O

Prima di entrambi i casi, il kernel registra gli errori I/O che innescano fserror_report. Questi messaggi sono originali, non aggiunti dal debug:

LOG — log_debug.txt righe 1736-1737 — originali (kernel)

```
[ 246.281367][ C0] I/O error, dev loop0, sector 0 op 0x0:(READ)
[ 246.282481][ C0] I/O error, dev loop0, sector 18692 op 0x0:(READ)
```

Subito dopo, l'esecuzione si biforca: il primo errore è gestito da ksoftirqd (caso asincrono), il secondo dal processo repro (caso sincrono).

3.2. Il warning lockdep — originale, non modificato

Il warning lockdep (righe 1767-1843) è stato prodotto dalla macchina lockdep del kernel **senza alcuna modifica al codice**. È la stessa segnalazione che appare nel bug report originale di syzbot. Lockdep tiene traccia di come ogni classe di lock viene usata nel sistema e segnala quando un utilizzo è incompatibile con un utilizzo precedente.

LOG — log_debug.txt righe 1768-1773 — intestazione warning (originale)

```
WARNING: inconsistent lock state
inconsistent {SOFTIRQ-ON-W} -> {IN-SOFTIRQ-W} usage.
ksoftirqd/0/15 [HC0[0]:SC1[1]:HE1:SE0] takes:
ffff88801d0f1e80 (&sb->s_type->i_lock_key#34){+..?..}-{3:3},
at: igrab+0x21/0x140
```

Lettura riga per riga:

- {SOFTIRQ-ON-W}: questa classe di lock (i_lock_key#34, cioè il lock degli inode XFS) era già stata acquisita in **write** con le softirq **abilitate** — registrato da lockdep al momento del mount.
- {IN-SOFTIRQ-W}: ora viene acquisita in **write** mentre si è **dentro** una softirq — questo è il problema.
- at: igrab+0x21/0x140: il punto esatto della violazione, dentro igrab.

LOG — log_debug.txt righe 1774-1786 — dove era stato registrato lo stato SOFTIRQ-ON-W

```
{SOFTIRQ-ON-W} state was registered at:
lock_acquire · _raw_spin_lock
unlock_new_inode
xfs_iget
xfs_mountfs ← al momento del mount di XFS
xfs_fs_fill_super · get_tree_bdev_flags · vfs_get_tree
path_mount · __x64_sys_mount
```

Lockdep aveva visto i_lock acquisito durante il mount di XFS, con le softirq abilitate (process context normale). Il fatto che ora lo stesso lock venga acquisito da dentro una softirq crea un'inconsistenza: se un thread in process context stesse tenendo i_lock nel momento in cui scatta una softirq sulla stessa CPU, la softirq tenterrebbe di acquisire lo stesso lock già detenuto → **deadlock**.

LOG — log_debug.txt righe 1794-1802 — scenario di deadlock ipotetico

```
Possible unsafe locking scenario:

CPU0
--
lock(&sb->s_type->i_lock_key#34); ← process context, softirq abilitate

lock(&sb->s_type->i_lock_key#34); ← softirq tenta stesso lock

*** DEADLOCK ***
```

Il deadlock non si è manifestato concretamente (non ci sono lock held da ksoftirqd in quel momento), ma la sequenza di acquisizioni lo rende strutturalmente possibile — e lockdep lo segnala preventivamente.

3.3. I messaggi DBG — aggiunti per catturare lo stato

I messaggi [DBG] **non** sono presenti nel kernel originale. Sono stati aggiunti in iomap_finish_folio_read e in fserror_report per catturare lo stato dell'inode (i_count, i_state, in_softirq) nel momento esatto della chiamata, fornendo la **prova sperimentale** che accompagna la prova formale.

3.3.1. Caso asincrono — ksoftirqd (pid=15)

LOG — log_debug.txt righe 1738-1740 — DBG aggiunti, caso asincrono

```
[C0] iomap_finish_folio_read [DBG]
error=-5 ino=9286 i_count=1 i_state=0x0
(I_FREEING=0 I_WILL_FREE=0) in_softirq=256 pid=15 comm=ksoftirqd/0

[C0] fserror_report [DBG] #1 entry:
in_softirq=256 in_interrupt=256 pid=15 comm=ksoftirqd/0

[C0] fserror_report [DBG] #1 inode:
ino=9286 i_count=1 i_state=0x0 (I_FREEING=0 I_WILL_FREE=0)
```

Tre informazioni chiave: i_count=1 (condizione necessaria soddisfatta), i_state=0x0 (flag tutti a zero), in_softirq=256 (conferma il contesto softirq che ha innescato il warning).

Subito dopo, il `dump_stack` interno a `fserror_report` produce la call trace che mostra l'intera catena asincrona (righe 1746–1757):

LOG — log_debug.txt righe 1746-1757 — call trace asincrona (da `dump_stack` in `fserror_report`)

```
fserror_report+0x4ed/0x580
iomap_finish_folio_read.cold+0x5b/0x76
iomap_read_end_io+0x16f/0x230
bio_endio+0x4d3/0x650
blk_update_request · blk_mq_end_request
lo_complete_rq
blk_done_softirq ← handler BLOCK_SOFTIRQ
handle_softirqs · run_ksoftirqd
```

A seguire, il warning `lockdep` (righe 1767–1843). Infine, dopo che `igrab` ha eseguito, il `DBG` finale mostra l'effetto dell'incremento:

LOG — log_debug.txt riga 1844 — `DBG` dopo `igrab` (aggiunto)

```
fserror_report [DBG] #1 after igrab: sb=loop0 ino=9286 i_count=2 (expected >= 2)
```

3.3.2. Caso sincrono — `repro` (`pid=5860`)

LOG — log_debug.txt righe 1845-1847 — `DBG` aggiunti, caso sincrono

```
[T5860] iomap_finish_folio_read [DBG]
error=-5 ino=9286 i_count=2 i_state=0x0
(I_FREEING=0 I_WILL_FREE=0) in_softirq=0 pid=5860 comm=repro

[T5860] fserror_report [DBG] #2 entry:
in_softirq=0 in_interrupt=0 pid=5860 comm=repro

[T5860] fserror_report [DBG] #2 inode:
ino=9286 i_count=2 i_state=0x0 (I_FREEING=0 I_WILL_FREE=0)
```

`in_softirq=0` conferma che si è in process context — per questo il warning non scatta. `i_count=2` perché il caso asincrono aveva già eseguito `igrab` (che ha portato `i_count` da 1 a 2).

La call trace sincrona (righe 1853–1883) mostra la differenza strutturale rispetto a quella asincrona:

LOG — log_debug.txt righe 1853-1883 — call trace sincrona, differenze chiave evidenziate

```
fserror_report · iomap_finish_folio_read · iomap_read_end_io · bio_endio
submit_bio_noacct ← bio completato inline, prima che ritorni
iomap_bio_submit_read · iomap_read_folio · filemap_read_folio
...
security_file_post_open
path_openat ← openat() ancora attiva sullo stack
__x64_sys_openat
```

Le due righe evidenziate mostrano le differenze strutturali rispetto al caso asincrono: `submit_bio_noacct` sullo stack (bio inline) e `path_openat` visibile in fondo (`openat` non ha ancora ritornato).

3.4. Diff delle due call trace

Entrambi i casi partono dallo stesso stack di processo (`repro`): `__x64_sys_openat` → ... → `submit_bio_noacct`. La biforcazione avviene **dentro** `submit_bio_noacct`: nel caso sincrono il loop driver completa il bio come chiamate annidate (inline), senza uscire dalla funzione; nel caso asincrono `submit_bio_noacct` ritorna subito e il processo si blocca su `io_schedule()`

— il completamento viene gestito da ksoftirqd su un thread separato. La catena di completamento (bio_endio → ... → fserror_report) è identica in entrambi i casi.

ASINCRONO — ksoftirqd/0 (pid=15)	SINCRONO — repro (pid=5860)
▲ PERCORSO COMUNE — catena di completamento bio ▲	
fserror_report+0x4ed/0x580	fserror_report+0x4ed/0x580
iomap_finish_folio_read.cold+0x5b/0x76	iomap_finish_folio_read.cold+0x5b/0x76
iomap_read_end_io+0x16f/0x230	iomap_read_end_io+0x16f/0x230
bio_endio+0x4d3/0x650	bio_endio+0x4d3/0x650
▼ BIFORCAZIONE ▼ — come bio_endio viene raggiunta	
blk_update_request+0x3f2/0x880	<i>bio completato inline</i> Nessun frame extra: bio_endio è chiamata direttamente dentro submit_bio_noacct ↓
blk_mq_end_request+0x27/0x1f0	
io_complete_rq+0xcb/0x130	
blk_done_softirq+0x62/0x90 ◀	
handle_softirqs+0xea/0x5d0	
run_ksoftirqd+0x38/0x60	
▼ PERCORSO COMUNE — stack del processo repro ▼ (asincrono: processo congelato su io_schedule() — stack immobile)	
submit_bio_noacct (processo bloccato)	submit_bio_noacct+0x35a/0xd30 ◀
iomap_bio_submit_read	iomap_bio_submit_read+0x23/0x30
iomap_read_folio	iomap_read_folio+0x1a3/0x340
filemap_read_folio	filemap_read_folio+0x7b/0x210
... (IMA: __kernel_read →	... (IMA: __kernel_read →
security_file_post_open)	security_file_post_open)
path_openat (processo bloccato)	path_openat+0xad2/0x17f0 ◀
__x64_sys_openat (processo bloccato)	__x64_sys_openat+0x82/0xf0

◀ = punto chiave. Lo stack inferiore (grigio/blu) è identico in entrambi i casi — è la catena di processo che ha sottomesso il bio. Nel caso asincrono quel processo è congelato e la catena superiore (verde) gira su ksoftirqd; nel caso sincrono le due sezioni formano un unico stack continuo.

3.5. Relazione tra log e prova formale

Il warning lockdep **identifica il problema** (igrab in softirq) ma non dice nulla su i_count — non è suo compito. I messaggi DBG **confermano sperimentalmente** che i_count ≥ 1 e i flag sono a 0 in entrambi i casi. La prova formale nelle sezioni successive spiega **perché strutturalmente** questo deve essere sempre vero — indipendentemente dal numero di iterazioni del reproducer.

4. Prova formale — espansione ricorsiva della call chain

La prova srotola ogni livello della call chain partendo da openat, sostituendo ogni chiamata con il corpo della funzione invocata, fino a fserror_report. I livelli L0-L5 sono comuni ad entrambi i casi; la biforcazione avviene dentro submit_bio_noacct (L5).

4.1. Percorso comune — da openat a submit_bio_noacct

4.1.1. L0 — __x64_sys_openat • arch/x86/entry/syscalls/syscall_64.tbl

```
// Entry point della syscall openat (#257 su x86-64).
SYSCALL_DEFINE4(openat, int, dfd, const char __user *, filename,
```

```

        int, flags, umode_t, mode)
{
    if (force_o_largefile())
        flags |= O_LARGEFILE;
    return do_sys_open(dfd, filename, flags, mode);    // ← L0a
}

```

4.1.2. L0a — do_sys_open · fs/open.c

```

int do_sys_open(int dfd, const char __user *filename, int flags, umode_t mode)
{
    struct open_how how = build_open_how(flags, mode);
    return do_sys_openat2(dfd, filename, &how);    // ← L0b
}

```

4.1.3. L0b — do_sys_openat2 · fs/open.c

```

static int do_sys_openat2(int dfd, const char __user *filename,
                          struct open_how *how)
{
    struct open_flags op;
    int err = build_open_flags(how, &op);
    if (unlikely(err))
        return err;
    CLASS(filename, name)(filename);
    return FD_ADD(how->flags, do_file_open(dfd, name, &op));    // ← L0c
}

```

4.1.4. L0c — do_file_open · fs/namei.c

```

struct file *do_file_open(int dfd, struct filename *pathname,
                          const struct open_flags *op)
{
    struct nameidata nd;
    int flags = op->lookup_flags;
    struct file *filp;

    set_nameidata(&nd, dfd, pathname, NULL);
    filp = path_openat(&nd, op, flags | LOOKUP_RCU);    // ← L1 (fast path RCU)
    if (unlikely(filp == ERR_PTR(-ECHILD)))
        filp = path_openat(&nd, op, flags);    // retry senza RCU
    if (unlikely(filp == ERR_PTR(-ESTALE)))
        filp = path_openat(&nd, op, flags | LOOKUP_REVAL);    // retry con reval
    restore_nameidata();
    return filp;
}

```

do_file_open chiama path_openat fino a tre volte, ma solo per gestire fallimenti del lookup RCU — in condizioni normali il primo path_openat ha successo. do_file_open non può ritornare finché path_openat non ritorna. La stessa attesa si propaga a ritroso fino a __x64_sys_openat.

4.1.5. L1 — path_openat · fs/namei.c

```

static struct file *path_openat(struct nameidata *nd,
                                const struct open_flags *op, unsigned flags)
{

```

```

struct file *file;
int error;

file = alloc_empty_file(op->open_flag, current_cred());
...
const char *s = path_init(nd, flags);
while (!(error = link_path_walk(s, nd)) &&
        (s = open_last_lookups(nd, file, op)) != NULL)    // ← L1a
    ;
// open_last_lookups ritorna NULL quando il lookup è completo,
// o una stringa (symlink da seguire) per iterare il while.
// i_count ≥ 1 è garantito prima di uscire dal while (vedi L1a-L1e).
if (!error)
    error = do_open(nd, file, op);    // → security_file_post_open ← L2
terminate_walk(nd);
...
// path_openat non ritorna finché do_open non ritorna
}

```

Il ciclo while gestisce i symlink: link_path_walk segue i componenti del path, poi open_last_lookups risolve l'ultimo componente. Se open_last_lookups ritorna una stringa non-NULL, il ciclo riparte per seguire il symlink. Al termine del ciclo (open_last_lookups → NULL), l'inode è stato individuato e i_count ≥ 1 (vedi L1a-L1e). Solo a quel punto viene chiamata do_open.

4.1.6. L1a — open_last_lookups · fs/namei.c

```

static const char *open_last_lookups(struct nameidata *nd,
                                     struct file *file, const struct open_flags *op)
{
    struct dentry *dentry;
    ...
    // PERCORSO A – dentry già in dcache (caso più comune):
    dentry = lookup_fast_for_open(nd, open_flag);
    // → __d_lookup_rcu / __d_lookup: cerca in hash table
    // → ritorna dentry con d_inode != NULL
    // → i_count già ≥ 1 (mantenuto dalla dentry esistente in dcache,
    //   che possiede il riferimento preso da xfs_iget al primo caricamento)
    if (likely(dentry))
        goto finish_lookup;    // ← salta lookup_open, non chiama xfs_iget

    // PERCORSO B – dentry non in cache (primo accesso o dentry evicted):
    dentry = lookup_open(nd, file, op, got_write, &delegated_inode);
    // → dir_inode->i_op->lookup(dir_inode, dentry, flags) ← L1b
    //   [= xfs_vn_lookup per filesystem XFS]
    // → xfs_iget → igrab → atomic_inc(&inode->i_count) ← L1c-L1e
    ...
finish_lookup:
    ...
    res = step_into(nd, WALK_TRAILING, dentry);
    return res;    // NULL = lookup completato; non-NULL = symlink da seguire
}

```

In entrambi i percorsi, quando open_last_lookups termina l'inode associato alla dentry ha i_count ≥ 1. Nel percorso A il riferimento preesiste; nel percorso B viene creato ora tramite igrab. Il livello L1b-L1e descrive il percorso B in dettaglio.

Approfondimento — perché la dentry garantisce i_count ≥ 1

La relazione dentry → inode non è una semplice osservazione: è un contratto esplicito del kernel, documentato nel sorgente stesso.

Quando una dentry viene istanziata (d_instantiate / d_splice_alias), il commento in fs/dcache.c recita: «NOTE! This assumes that the inode count has been incremented (caller's responsibility)». Questo significa che chiunque chiami d_instantiate deve già aver incrementato i_count (via iggrab o ihold). La dentry «eredita» quel riferimento.

Quando la dentry viene distrutta (__dentry_kill), viene chiamata dentry_unlink_inode che esegue iput(inode), bilanciando esattamente l'incremento iniziale. Il ciclo di vita è quindi:

```
xfs_iget → iggrab → atomic_inc(i_count)    [riferimento CREATO]
    └─ d_splice_alias / d_instantiate        [passato alla dentry]
        └─ dentry viva (d_count ≥ 1 o su LRU)
            └─ __dentry_kill
                └─ dentry_unlink_inode
                    └─ iput → atomic_dec(i_count) [riferimento RILASCIATO]
```

Punto critico: anche quando d_count (il refcount della dentry) raggiunge 0 tramite dput, la dentry NON viene distrutta immediatamente se è ancora nella hash table della dcache: viene messa sulla LRU (retain_dentry ritorna true). Finché è sulla LRU, d_inode != NULL e il riferimento i_count è ancora mantenuto. La dentry viene uccisa (e quindi iput chiamato) solo quando viene effettivamente evicted dalla LRU dal memory shrinker, oppure quando il filesystem la invalida esplicitamente.

Durante path_openat: la dentry è attivamente in uso tramite nd->path.dentry (con d_count ≥ 1). Non può essere sulla LRU e non può essere evicted. Quindi i_count ≥ 1 è garantito per tutta la durata della chiamata a path_openat, indipendentemente dal percorso A o B.

4.1.7. L1b — lookup_open → xfs_vn_lookup • fs/namei.c + fs/xfs/xfs_iops.c

```
// fs/namei.c
static struct dentry *lookup_open(struct nameidata *nd, struct file *file, ...)
{
    struct inode *dir_inode = dir->d_inode;
    ...
    dentry = d_lookup(dir, &nd->last);    // cerca ancora in hash table
    ...
    if (dentry->d_inode) {
        return dentry;    // dentry positiva trovata → già gestito
    }
    ...
    if (d_in_lookup(dentry)) {
        struct dentry *res = dir_inode->i_op->lookup(dir_inode, dentry,
                                                    nd->flags);
        // per XFS: i_op->lookup = xfs_vn_lookup    ← sotto
        ...
    }
}

// fs/xfs/xfs_iops.c
STATIC struct dentry *
xfs_vn_lookup(struct inode *dir, struct dentry *dentry, unsigned int flags)
{
    struct xfs_inode *cip;
    struct xfs_name name;
    int error;

    xfs_dentry_to_name(&name, dentry);
    error = xfs_lookup(XFS_I(dir), &name, &cip, NULL);    // ← L1c
```

```

    if (likely(!error))
        inode = VFS_I(cip);
    ...
    return d_splice_alias(inode, dentry);
    // d_splice_alias associa l'inode alla dentry;
    // il riferimento preso da xfs_iget è ora "posseduto" dalla dentry.
}

```

4.1.8. L1c — xfs_lookup · fs/xfs/xfs_inode.c

```

int
xfs_lookup(struct xfs_inode *dp, const struct xfs_name *name,
           struct xfs_inode **ipp, struct xfs_name *ci_name)
{
    xfs_ino_t inum;
    int error;

    if (xfs_is_shutdown(dp->i_mount))
        return -EIO;

    error = xfs_dir_lookup(NULL, dp, name, &inum, ci_name);
    // ricerca del nome nella directory XFS → ottiene il numero di inode
    if (error)
        goto out_unlock;

    error = xfs_iget(dp->i_mount, NULL, inum, 0, 0, ipp);    // ← L1d
    ...
}

```

4.1.9. L1d — xfs_iget → xfs_iget_cache_hit · fs/xfs/xfs_icache.c

```

int
xfs_iget(struct xfs_mount *mp, struct xfs_trans *tp, xfs_ino_t ino,
         uint flags, uint lock_flags, struct xfs_inode **ipp)
{
    struct xfs_inode *ip;
    struct xfs_perag *pag;
    ...
    pag = xfs_perag_get(mp, XFS_INO_TO_AGNO(mp, ino));
    agino = XFS_INO_TO_AGINO(mp, ino);
again:
    rcu_read_lock();
    ip = radix_tree_lookup(&pag->pag_ici_root, agino);

    if (ip) {
        error = xfs_iget_cache_hit(pag, ip, ino, flags, lock_flags);
        // ← L1e: inode trovato nella radix tree XFS, chiama igrab
    } else {
        error = xfs_iget_cache_miss(mp, pag, tp, ino, &ip, ...);
        // inode non in cache: alloca e carica da disco, poi chiama igrab
    }
    ...
    *ipp = ip;    // ritorna l'inode con i_count incrementato
}

static int
xfs_iget_cache_hit(struct xfs_perag *pag, struct xfs_inode *ip,
                  xfs_ino_t ino, int flags, int lock_flags)
{
    struct inode *inode = VFS_I(ip);
}

```

```

...
spin_lock(&ip->i_flags_lock);
...
if (ip->i_flags & (XFS_INEW | XFS_IRECLAIM | XFS_INACTIVATING))
    goto out_skip;    // inode in stato transitorio, riprova
...
if (!igrab(inode))    // ← L1e: incrementa i_count se I_FREEING non set
    goto out_skip;

spin_unlock(&ip->i_flags_lock);
rcu_read_unlock();
...
return 0;    // successo: *ipp ha i_count ≥ 1
}

```

4.1.10. L1e — igrab → __iget · fs/inode.c + include/linux/fs.h

```

// fs/inode.c
struct inode *igrab(struct inode *inode)
{
    spin_lock(&inode->i_lock);
    if (!(inode_state_read(inode) & (I_FREEING | I_WILL_FREE))) {
        __iget(inode);    // ← atomic_inc
        spin_unlock(&inode->i_lock);
    } else {
        spin_unlock(&inode->i_lock);
        inode = NULL;    // inode in fase di liberazione: fallisce
    }
    return inode;
}

// include/linux/fs.h
static inline void __iget(struct inode *inode)
{
    lockdep_assert_held(&inode->i_lock);
    atomic_inc(&inode->i_count);    // ← i_count portato a ≥ 1
}

```

igrab verifica atomicamente (con *i_lock*) che *I_FREEING* e *I_WILL_FREE* siano entrambi zero prima di incrementare *i_count*. Questo è esattamente il controllo inverso rispetto alla preconditione che vogliamo provare: se *igrab* ha avuto successo, allora al momento dell'incremento *I_FREEING* = *I_WILL_FREE* = 0.

Il riferimento preso da *igrab* qui (percorso B) viene poi «trasferito» alla dentry tramite *d_splice_alias*. Da quel momento in poi, la garanzia *i_count* ≥ 1 è mantenuta dalla dentry stessa (vedi approfondimento sopra), non più da un holder esplicito nella call stack.

4.1.11. L2 — do_open → security_file_post_open · fs/namei.c

```

static int do_open(struct nameidata *nd,
                  struct file *file, const struct open_flags *op)
{
    ...
    error = vfs_open(&nd->path, file);
    if (!error)
        error = security_file_post_open(file, op->acc_mode);
    // LSM hook → ima_file_check → ima_check_last_writer
    // → __kernel_read    ← L3
}

```

```

    ...
}

```

4.1.12. L3 — __kernel_read · fs/read_write.c

```

ssize_t __kernel_read(struct file *file, void *buf, size_t count, loff_t *pos)
{
    struct kiocb kiocb;
    struct iov_iter iter;
    ...
    init_sync_kiocb(&kiocb, file);
    kiocb.ki_pos = pos ? *pos : 0;
    iov_iter_kvec(&iter, ITER_DEST, &iov, 1, iov.iov_len);
    ret = file->f_op->read_iter(&kiocb, &iter);    // XFS → ← L3a
    ...
}

```

4.1.13. L3a — xfs_file_read_iter · fs/xfs/xfs_file.c

```

STATIC ssize_t
xfs_file_read_iter(struct kiocb *iocb, struct iov_iter *to)
{
    struct inode *inode = file_inode(iocb->ki_filp);
    struct xfs_mount *mp = XFS_I(inode)->i_mount;
    ssize_t ret = 0;

    if (xfs_is_shutdown(mp))
        return -EIO;

    if (IS_DAX(inode))
        ret = xfs_file_dax_read(iocb, to);
    else if (iocb->ki_flags & IOCB_DIRECT)
        ret = xfs_file_dio_read(iocb, to);
    else
        ret = xfs_file_buffered_read(iocb, to);    // ← L3b (percorso IMA: buffered)
    ...
}

```

4.1.14. L3b — xfs_file_buffered_read · fs/xfs/xfs_file.c

```

STATIC ssize_t
xfs_file_buffered_read(struct kiocb *iocb, struct iov_iter *to)
{
    struct xfs_inode *ip = XFS_I(file_inode(iocb->ki_filp));
    ssize_t ret;

    ret = xfs_ilock_iocb(iocb, XFS_IOLOCK_SHARED);    // acquisisce i-lock
    if (ret)
        return ret;
    ret = generic_file_read_iter(iocb, to);            // ← L3c
    xfs_iunlock(ip, XFS_IOLOCK_SHARED);
    return ret;
}

```

4.1.15. L3c — generic_file_read_iter · mm/filemap.c

```

ssize_t generic_file_read_iter(struct kiocb *iocb, struct iov_iter *iter)
{
    ...
    if (iocb->ki_flags & IOCB_DIRECT) {
        ... // percorso DIO, non preso per IMA
    }
    return filemap_read(iocb, iter, retval); // ← L3d (percorso buffered)
}

```

4.1.16. L3d — filemap_read · mm/filemap.c

```

ssize_t filemap_read(struct kiocb *iocb, struct iov_iter *iter,
                    ssize_t already_read)
{
    ...
    do {
        ...
        error = filemap_get_pages(iocb, iter->count, &fbatch, false);
        // ↑ se il folio non è uptodate → filemap_update_page ← L3e
        ...
    } while (iov_iter_count(iter) && iocb->ki_pos < isize && !error);
    ...
}

```

4.1.17. L3e — filemap_get_pages → filemap_update_page · mm/filemap.c

```

static int filemap_get_pages(struct kiocb *iocb, size_t count,
                            struct folio_batch *fbatch, bool need_uptodate)
{
    ...
    filemap_get_read_batch(mapping, index, last_index - 1, fbatch);
    // cerca i folio già in page cache nel range richiesto

    if (!folio_batch_count(fbatch)) {
        page_cache_sync_ra(&ractl, last_index - index);
        // nessun folio in cache: avvia readahead sincrono
        filemap_get_read_batch(mapping, index, last_index - 1, fbatch);
    }
    if (!folio_batch_count(fbatch)) {
        err = filemap_create_folio(iocb, fbatch);
        // ancora nessun folio: alloca un nuovo folio in cache
        ...
    }

    folio = fbatch->folios[folio_batch_count(fbatch) - 1];
    if (folio_test_readahead(folio)) {
        err = filemap_readahead(iocb, filp, mapping, folio, last_index);
        ...
    }
    if (!folio_test_uptodate(folio)) {
        // folio presente ma NON aggiornato (dati non ancora letti):
        err = filemap_update_page(iocb, mapping, count, folio,
                                need_uptodate); // ← L4 via L3e-bis
        ...
    }
    return 0;
}

static int filemap_update_page(struct kiocb *iocb,
                              struct address_space *mapping, size_t count,

```

```

    struct folio *folio, bool need_uptodate)
{
    ...
    if (!folio_trylock(folio)) {
        // folio bloccato da I/O in corso → aspetta
        folio_put_wait_locked(folio, TASK_KILLABLE);
        return AOP_TRUNCATED_PAGE;    // riprova il loop in filemap_get_pages
    }
    ...
    if (filemap_range_uptodate(...))
        goto unlock;    // folio già aggiornato nel frattempo

    // folio bloccato, non uptodate, nessun flag NOIO/NOWAIT:
    error = filemap_read_folio(iocb->ki_filp,
                              mapping->a_ops->read_folio,
                              folio);    // ← L4
    ...
}

```

La catena corretta è filemap_read → filemap_get_pages → filemap_update_page → filemap_read_folio. filemap_get_pages non chiama filemap_read_folio direttamente. filemap_update_page si interpone: verifica il lock del folio, controlla se nel frattempo è diventato uptodate, e solo allora invoca filemap_read_folio.

4.1.18. L4 — filemap_read_folio · mm/filemap.c — punto di sospensione (caso asincrono)

```

static int filemap_read_folio(struct file *file, filler_t filler,
                             struct folio *folio)
{
    ...
    error = filler(file, folio);
    // filler = iomap_read_folio → iomap_bio_submit_read
    // → submit_bio_noacct    ← L5 / BIFORCAZIONE
    //
    // ASYNC: filler ritorna subito dopo aver messo il bio in coda.
    // SYNC: filler non ritorna finché bio_endio non ha completato inline.

    if (error)
        return error;
    error = folio_wait_locked_killable(folio); // (C) ← ASYNC: processo sospeso qui
    ...                                     // finché PG_locked = 1
}

```

Punto di sospensione (caso asincrono): nel caso async filler ritorna subito e il processo si blocca in folio_wait_locked_killable (C). PG_locked viene azzerato soltanto da folio_end_read — il passo (B) in iomap_finish_folio_read, che esegue **dopo** fserror_report_io (A). Finché (A) è in esecuzione, il processo è fermo in (C) con l'intera catena L0-L4 immobile sullo stack.

4.1.19. L5 — submit_bio_noacct · block/blk-core.c — punto di biforcazione

```

void submit_bio_noacct(struct bio *bio)
{
    ...
    // Il loop device sceglie uno dei due percorsi:
    //
    // CASO ASINCRONO —————

```

```

// Il backing file non è in page cache: il bio viene messo in coda.
// submit_bio_noacct RITORNA SUBITO al chiamante (filler).
// Il processo torna a L4 e si blocca in folio_wait_locked_killable (C).
// Il completamento avverrà su ksoftirqd → rami LA
//
// CASO SINCRONO —————
// Il backing file è in page cache: il loop driver esegue il bio
// inline. bio_endio viene chiamata DENTRO questa funzione, prima
// che submit_bio_noacct ritorni. → rami LS
...
}

```

4.2. Caso asincrono — completamento su ksoftirqd (rami LA)

Il processo è bloccato in (C) a L4. Su una CPU separata, ksoftirqd/0 gestisce il BLOCK_SOFTIRQ accodato al termine dell'I/O.

4.2.1. LA1 — run_ksoftirqd → handle_softirqs • kernel/softirq.c

```

static void run_ksoftirqd(unsigned int cpu)
{
    ksoftirqd_run_begin();
    if (local_softirq_pending()) {
        handle_softirqs(true); // processa i softirq pendenti
    }
    ksoftirqd_run_end();
}

static void handle_softirqs(bool ksoftirqd)
{
    ...
    while ((softirq_bit = ffs(pending))) {
        h += softirq_bit - 1;
        h->action(); // BLOCK_SOFTIRQ → blk_done_softirq ← LA2
        ...
    }
}

```

4.2.2. LA2 — blk_done_softirq • block/blk-mq.c

```

// Registrata come handler del BLOCK_SOFTIRQ in blk_mq_init():
// open_softirq(BLOCK_SOFTIRQ, blk_done_softirq)

static void blk_complete_reqs(struct llist_head *list)
{
    struct llist_node *entry = llist_reverse_order(llist_del_all(list));
    struct request *rq, *next;

    llist_for_each_entry_safe(rq, next, entry, ipi_list)
        rq->q->mq_ops->complete(rq); // → lo_complete_rq ← LA3
}

static __latent_entropy void blk_done_softirq(void)
{
    blk_complete_reqs(this_cpu_ptr(&blk_cpu_done));
}

```

4.2.3. LA3 — lo_complete_rq • drivers/block/loop.c

```
static void lo_complete_rq(struct request *rq)
{
    struct loop_cmd *cmd = blk_mq_rq_to_pdu(rq);
    blk_status_t ret = cmd->ret < 0 ? BLK_STS_IOERR : BLK_STS_OK;
    ...
    blk_mq_end_request(rq, ret);    // ← LA3a
}
```

4.2.4. LA3a — blk_mq_end_request • block/blk-mq.c

```
void blk_mq_end_request(struct request *rq, blk_status_t error)
{
    if (blk_update_request(rq, error, blk_rq_bytes(rq)))
        BUG();
    __blk_mq_end_request(rq, error);
    // blk_update_request → ... → bio_endio    ← LA3b → LA4
}
```

blk_mq_end_request chiama blk_update_request passando il numero totale di byte della richiesta. È blk_update_request che, scorrendo i bio collegati, chiama bio_endio.

4.2.5. LA3b — blk_update_request • block/blk-mq.c

```
bool blk_update_request(struct request *req, blk_status_t error,
    unsigned int nr_bytes)
{
    ...
    while (req->bio) {
        struct bio *bio = req->bio;
        unsigned bio_bytes = min(bio->bi_iter.bi_size, nr_bytes);

        if (unlikely(error))
            bio->bi_status = error;
        ...
        bio_advance(bio, bio_bytes);

        if (!bio->bi_iter.bi_size) {
            req->bio = bio->bi_next;
            bio->bi_next = NULL;
            if (!is_flush)
                bio_endio(bio);    // ← LA4
        }
        ...
    }
    ...
    return false;
}
```

Quando bi_size raggiunge zero (tutti i byte del bio sono stati processati), blk_update_request chiama bio_endio(bio), che invoca il callback bi_end_io registrato — nel nostro caso iomap_read_end_io.

4.2.6. LA4 — bio_endio → iomap_read_end_io • block/bio.c


```

void bio_endio(struct bio *bio)
{
    ...
    bio->bi_end_io(bio);    // bi_end_io registrato = iomap_read_end_io
}

// fs/iomap/bio.c
static void iomap_read_end_io(struct bio *bio)
{
    int error = blk_status_to_errno(bio->bi_status);
    struct folio_iter fi;

    bio_for_each_folio_all(fi, bio)
        iomap_finish_folio_read(fi.folio, fi.offset, fi.length, error); // ← LA5
    bio_put(bio);
}

```

4.2.7. LA5 — iomap_finish_folio_read · fs/iomap/buffered-io.c — termine della catena asincrona

```

void iomap_finish_folio_read(struct folio *folio, size_t off,
                             size_t len, int error)
{
    bool uptodate = !error;
    bool finished;
    ...
    if (error) // gestione ifs (iomap_folio_state)
        fserror_report_io(folio->mapping->host, FSERR_BUFFERED_READ,
                           folio_pos(folio) + off, len, error, GFP_ATOMIC);
    // (A) ← fserror_report in esecuzione
    //      PG_locked = 1
    //      processo bloccato in (C) a L4

    if (finished)
        folio_end_read(folio, uptodate);
    // (B) ← eseguito solo DOPO (A)
    //      azzera PG_locked → sblocca (C) → processo riprende da L4
}

```

Caso asincrono — $i_count \geq 1$: (A) precede (B) nel sorgente → mentre fserror_report è in esecuzione PG_locked = 1 → processo bloccato in folio_wait_locked_killable (C) a L4 → l'intera catena L0-L4 è immobile sullo stack → path_openat (L1) non ha ritornato → openat() attiva → close() non avvenuta → $i_count \geq 1$. □

LOG — log_debug.txt — call trace asincrona (righe 1746-1757) e DBG (riga 1738)

```

ksoftirqd/0 (pid=15) ← stack al momento di fserror_report:
fserror_report (LA5) · iomap_finish_folio_read (LA5) · iomap_read_end_io (LA4)
bio_endio (LA4) · lo_complete_rq (LA3) · blk_done_softirq (LA2)
handle_softirqs (LA1) · run_ksoftirqd (LA1)

DBG: i_count=1 i_state=0x0 (I_FREEING=0 I_WILL_FREE=0)

```

4.3. Caso sincrono — completamento inline (rami LS)

Il bio viene completato inline dentro submit_bio_noacct. Il processo non si ferma mai in folio_wait_locked_killable: i rami LS fanno parte dello stesso stack continuo che va da L0 a fserror_report.

4.3.1. LS1 — submit_bio_noacct • block/blk-core.c

```
void submit_bio_noacct(struct bio *bio)
{
    struct block_device *bdev = bio->bi_bdev;
    struct request_queue *q = bdev_get_queue(bdev);
    blk_status_t status = BLK_STS_IOERR;    // ← inizializzato a IOERR

    might_sleep();

    if ((bio->bi_opf & REQ_NOWAIT) && !bdev_nowait(bdev))
        goto not_supported;                // → BLK_STS_NOTSUPP ≠ IOERR

    if (bio_has_crypt_ctx(bio)) {
        if (WARN_ON_ONCE(!bio_has_data(bio)))
            goto end_io;                    // WARN splat nel log
        ...
    }

    if (should_fail_bio(bio))                // ← (A) fault injection silente
        goto end_io;

    bio_check_ro(bio);                       // solo warning, non goto
    if (!bio_flagged(bio, BIO_REMAPPED)) {
        if (unlikely(bio_check_eod(bio)))
            goto end_io;                    // (B) pr_info_ratelimited nel log
        if (bdev_is_partition(bdev) &&
            unlikely(blk_partition_remap(bio)))
            goto end_io;                    // (C) solo per partizioni
    }

    if (op_is_flush(bio->bi_opf)) {
        if (WARN_ON_ONCE(...))
            goto end_io;                    // WARN splat nel log
        ...
    }

    switch (bio_op(bio)) {
    case REQ_OP_READ:
        break;                             // ← il nostro bio: nessun goto
        ...
    }

    submit_bio_noacct_nocheck(bio, false);   // path normale, accodato
    return;

not_supported:
    status = BLK_STS_NOTSUPP;
end_io:
    bio->bi_status = status;                 // BLK_STS_IOERR → errno = -EIO = -5
    bio_endio(bio);                         // ← LS2: INLINE nel thread del processo
}
```

Prova per eliminazione — perché il path attivo è should_fail_bio

Il trace mostra submit_bio_noacct+0x35a → bio_endio CON bio->bi_status = BLK_STS_IOERR (= errno -5 = -EIO). status è inizializzato a BLK_STS_IOERR e cambiato solo in due posti: BLK_STS_OK (flush senza settori) e BLK_STS_NOTSUPP (not_supported:). Nessuno dei due vale qui. Quindi il valore -EIO proviene dall'inizializzazione e il goto end_io non ha cambiato status. Elenco di tutti i goto end_io con BLK_STS_IOERR invariato e motivo di esclusione:

- **(A) WARN_ON_ONCE(!bio_has_data) dentro bio_has_crypt_ctx:** produrrebbe un kernel WARN splat visibile nel log. Assente. **Escluso.**
- **(B) bio_check_eod:** emette pr_info_ratelimited("attempt to access beyond end of device"). Assente nel log. Inoltre il bio legge pos=0x0, len=0x800 su un filesystem XFS (dimensione certamente >> 0x800). **Escluso.**
- **(C) blk_partition_remap fallisce:** richiede bdev_is_partition(bdev). Il device è /dev/loop0 (whole device, non partizione). **Escluso.**
- **(D) WARN_ON_ONCE per op_is_flush:** richiede op_is_flush(bio->bi_opf) (REQ_PREFLUSH o REQ_FUA). Il bio è un REQ_OP_READ. **Escluso.**
- **(E) blk_validate_atomic_write_op_size:** ramo REQ_OP_WRITE con REQ_ATOMIC. Il bio è REQ_OP_READ. **Escluso.**
- **(F) blk_check_zone_append:** ramo REQ_OP_ZONE_APPEND. **Escluso.**

Rimane un solo candidato silente compatibile con REQ_OP_READ,
pos=0, len=0x800, nessun messaggio nel log:

```
if (should_fail_bio(bio)) goto end_io;
```

should_fail_bio è implementata tramite il framework fail_make_request (vedi static DECLARE_FAULT_ATTR(fail_make_request) in block/blk-core.c) ed è esplicitamente annotata con ALLOW_ERROR_INJECTION(should_fail_bio, ERRNO) per l'uso da parte di syzbot. Il meccanismo tipico è la scrittura di 1 in /proc/self/make-it-fail prima di eseguire l'I/O da iniettare.

*Il bio viene completato con -EIO direttamente nel corpo di submit_bio_noacct, nel thread del processo **repro**. Nessun workqueue, nessun softirq. Per la semantica del C, submit_bio_noacct non può ritornare finché bio_endio (e tutte le funzioni da essa chiamate) non è ritornata → fserror_report ancora in esecuzione → submit_bio_noacct non ha ritornato → lo stesso vale per ogni chiamante nella catena fino a __x64_sys_openat.*

4.3.2. LS2 — bio_endio → iomap_read_end_io (stesso percorso di LA4)

```
// Percorso identico al caso asincrono – la differenza è che questi
// frame sono parte dello stack del processo repro, non di ksoftirqd.

void bio_endio(struct bio *bio) { bio->bi_end_io(bio); }

static void iomap_read_end_io(struct bio *bio) {
    bio_for_each_folio_all(fi, bio)
        iomap_finish_folio_read(...);    // ← LS3
}
```

4.3.3. LS3 — iomap_finish_folio_read (stesso sorgente di LA5)

```
void iomap_finish_folio_read(struct folio *folio, size_t off,
                             size_t len, int error)
{
    ...
    if (error)
        fserror_report_io(folio->mapping->host, FSERR_BUFFERED_READ,
                           folio_pos(folio) + off, len, error, GFP_ATOMIC);
    // (A) ← in esecuzione
    // submit_bio_noacct (LS1) non ha ritornato
    // filemap_read_folio (L4) non ha ritornato
    // path_openat (L1) non ha ritornato

    if (finished)
```

```

folio_end_read(folio, uptodate);
// (B) ← solo dopo (A); azzerà PG_locked
// poi submit_bio_noacct ritorna
// poi filemap_read_folio chiama folio_wait_locked_killable:
// PG_locked già 0, ritorna subito senza bloccare
}

```

Caso sincrono — $i_count \geq 1$: `ferror_report` è in esecuzione → (A) non ha ritornato → `iomap_finish_folio_read` non ha ritornato → `submit_bio_noacct` (LS1) non ha ritornato → per semantica del C ogni chiamante nella catena `LS1-LS3 ∪ L0-L5` è in attesa → `path_openat` (L1) non ha ritornato → `openat()` attiva → `close()` non avvenuta → $i_count \geq 1$. □

LOG — `log_debug.txt` — call trace sincrona (righe 1853-1883) e DBG (riga 1845)

```

repro (pid=5860) ← stack continuo L0→LS3:
ferror_report (LS3) · iomap_finish_folio_read (LS3) · iomap_read_end_io (LS2)
bio_endio (LS2) · submit_bio_noacct (LS1) · iomap_bio_submit_read
iomap_read_folio (L4) · filemap_read_folio (L4) · __kernel_read (L3)
security_file_post_open (L2) · path_openat (L1) · __x64_sys_openat (L0)

DBG: i_count=2 i_state=0x0 (I_FREEING=0 I_WILL_FREE=0)

```

Nota: $i_count=2$ nel caso sincrono. Il caso asincrono era già avvenuto: `igrab` (codice buggy) aveva portato i_count da 1 a 2. Quando arriva il caso sincrono il contatore vale già 2. La condizione è $i_count \geq 1$; $2 \geq 1$. □

5. `I_FREEING = 0` e `I_WILL_FREE = 0` (entrambi i casi)

`I_FREEING` e `I_WILL_FREE` vengono settati esclusivamente in `iput_final` (`fs/inode.c:1922`), protetta da una guardia esplicita:

```

// fs/inode.c:1922
static void iput_final(struct inode *inode)
{
    VFS_BUG_ON_INODE(atomic_read(&inode->i_count) != 0, inode); // guardia
    ...
    if (drop)
        inode_state_set(inode, I_FREEING); // unico punto nel kernel
    else
        inode_state_set(inode, I_WILL_FREE); // unico punto nel kernel
    ...
}

```

`iput_final` può essere raggiunta solo quando $i_count == 0$. Abbiamo dimostrato che in entrambi i casi $i_count \geq 1$ → `iput_final` non è stata invocata → `I_FREEING = I_WILL_FREE = 0`. □

6. Conclusione

Teorema: in entrambe le varianti della call chain, quando `ferror_report` è in esecuzione vale strutturalmente $i_count \geq 1$ e `I_FREEING = I_WILL_FREE = 0`.

L'iget che garantisce la condizione si trova nel **percorso comune**: `path_openat` → `open_last_lookups` → `xfs_iget` → `__iget` → `atomic_inc`, prima ancora che il controllo raggiunga IMA o il block layer.

- **Caso asincrono:** `ferror_report_io` (A) precede `folio_end_read` (B) nel sorgente; (B) è l'unico evento che sblocca il processo; durante (A) il processo è ancora bloccato con `openat()` attiva → `i_count ≥ 1`.
- **Caso sincrono:** `path_openat` è ancora sullo stack → `openat()` non ha ritornato → `close()` non avvenuta → `i_count ≥ 1`.
- **Flag:** `iput_final` (unico punto che setta i flag) richiede `i_count == 0`. Poiché `i_count ≥ 1`, i flag sono a 0.

La sostituzione `igrab` → `ihold` è formalmente corretta ed elimina l'acquisizione di `i_lock` da contesto `softirq` che causava il lockdep violation `{SOFTIRQ-ON-W}` → `{IN-SOFTIRQ-W}`.

Francesco Caligiuri · Kernel Linux for-next commit 2ffc6900d5c3 · Bug syzbot extid 5eb0d61dfb76ca12670c